

Phase 2: BCI & BTL API Documentation

Table of Contents

1. [BCI \(Binary Counting Interface\)](#)
 2. [BTL \(Boolean Translation Layer\)](#)
 3. [Build System](#)
 4. [Testing](#)
 5. [Performance](#)
-

BCI (Binary Counting Interface)

Binary Arithmetic Library (`bci_arithmetic.h`)

Provides multi-width binary arithmetic operations with carry and overflow detection.

Data Structures

```
typedef struct {
    uint64_t sum;
    bool carry;
} BinaryAddResult;

typedef struct {
    uint64_t product_low;
    uint64_t product_high;
    bool overflow;
} BinaryMulResult;
```

Functions

Addition with Carry

```
BinaryAddResult binary_add_u8(uint8_t a, uint8_t b, bool carry_in);
BinaryAddResult binary_add_u16(uint16_t a, uint16_t b, bool carry_in);
BinaryAddResult binary_add_u32(uint32_t a, uint32_t b, bool carry_in);
BinaryAddResult binary_add_u64(uint64_t a, uint64_t b, bool carry_in);
```

Multiplication with Overflow Detection

```
BinaryMulResult binary_mul_u8(uint8_t a, uint8_t b);
BinaryMulResult binary_mul_u16(uint16_t a, uint16_t b);
BinaryMulResult binary_mul_u32(uint32_t a, uint32_t b);
BinaryMulResult binary_mul_u64(uint64_t a, uint64_t b);
```

128-bit Operations (if `__SIZEOF_INT128__` is defined)

```
BinaryAddResult128 binary_add_u128(uint128_t a, uint128_t b, bool carry_in);
BinaryMulResult128 binary_mul_u128(uint128_t a, uint128_t b);
```

Example Usage

```
#include "bci/bci_arithmetic.h"

// Addition with carry
BinaryAddResult result = binary_add_u32(0xFFFFFFFF, 1, false);
if (result.carry) {
    printf("Overflow occurred! Sum: %u\n", result.sum);
}

// Multiplication with overflow detection
BinaryMulResult mul_result = binary_mul_u64(0xFFFFFFFFFFFFFFFF, 2);
if (mul_result.overflow) {
    printf("Product high: %llu, low: %llu\n",
        mul_result.product_high, mul_result.product_low);
}
```

Bit Manipulation Utilities (bci_bitops.h)

Comprehensive bit-level operations using compiler intrinsics with software fallbacks.

Bit Operations

```
uint64_t bit_set(uint64_t value, unsigned int bit);
uint64_t bit_clear(uint64_t value, unsigned int bit);
uint64_t bit_toggle(uint64_t value, unsigned int bit);
bool bit_test(uint64_t value, unsigned int bit);
```

Bit Counting

```
int popcount_u32(uint32_t value); // Count set bits
int popcount_u64(uint64_t value);

int clz_u32(uint32_t value);      // Count leading zeros
int clz_u64(uint64_t value);

int ctz_u32(uint32_t value);      // Count trailing zeros
int ctz_u64(uint64_t value);

int ffs_u32(uint32_t value);      // Find first set (1-indexed)
int ffs_u64(uint64_t value);

int parity_u32(uint32_t value);   // Parity of set bits
int parity_u64(uint64_t value);
```

Bit Manipulation

```
uint32_t bit_reverse_u32(uint32_t value);
uint64_t bit_reverse_u64(uint64_t value);

uint16_t bswap_u16(uint16_t value); // Byte swap
uint32_t bswap_u32(uint32_t value);
uint64_t bswap_u64(uint64_t value);
```

Example Usage

```
#include "bci/bci_bitops.h"

uint64_t flags = 0;
flags = bit_set(flags, 5);           // Set bit 5
flags = bit_toggle(flags, 3);        // Toggle bit 3

int count = popcount_u64(0xFF);      // Returns 8
int leading = clz_u32(0x00000001);   // Returns 31
int trailing = ctz_u32(0x00000008);  // Returns 3

uint32_t reversed = bit_reverse_u32(0x12345678);
uint32_t swapped = bswap_u32(0x12345678); // Endianness conversion
```

Conversion Utilities (`bci_conversion.h`)

Convert between binary, decimal, hexadecimal, and binary string representations.

Functions

```
// Binary ↔ Decimal
size_t binary_to_decimal_str(uint64_t value, char *buffer, size_t buffer_size);
bool decimal_str_to_binary(const char *str, uint64_t *out_value);

// Binary ↔ Hexadecimal
size_t binary_to_hex_str(uint64_t value, char *buffer, size_t buffer_size, bool upper-
case);
bool hex_str_to_binary(const char *str, uint64_t *out_value);

// Binary ↔ Binary String
size_t binary_to_binstr(uint64_t value, char *buffer, size_t buffer_size, int bits);
bool binstr_to_binary(const char *str, uint64_t *out_value);

// Binary ↔ Octal
size_t binary_to_octal_str(uint64_t value, char *buffer, size_t buffer_size);
bool octal_str_to_binary(const char *str, uint64_t *out_value);

// Formatted binary string with separators
size_t binary_to_binstr_formatted(uint64_t value, char *buffer, size_t buffer_size,
int bits, char separator, int group_size);
```

Example Usage

```
#include "bci/bci_conversion.h"

char buffer[128];

// Binary to hex
binary_to_hex_str(0xDEADBEEF, buffer, sizeof(buffer), true);
printf("%s\n", buffer); // "0xDEADBEEF"

// Hex to binary
uint64_t value;
hex_str_to_binary("0xCAFEBAFE", &value);

// Binary string representation
binary_to_binstr(0xAA, buffer, sizeof(buffer), 8);
printf("%s\n", buffer); // "0b10101010"

// Formatted binary with separators
binary_to_binstr_formatted(0xFFFF, buffer, sizeof(buffer), 16, '_', 4);
printf("%s\n", buffer); // "0b1111_1111_1111_1111"
```

SIMD Operations (`bci_simd.h`)

AVX2-accelerated vectorized binary operations with scalar fallbacks.

Feature Detection

```
bool bci_has_avx2_support(void); // Runtime CPU feature detection
```

AVX2 Operations (if `BCI_HAS_AVX2` is defined)

```
// 32-bit operations (8 elements at a time)
void binary_add_vec_avx2(const uint32_t *a, const uint32_t *b,
                        uint32_t *result, size_t count);
void binary_xor_vec_avx2(const uint32_t *a, const uint32_t *b,
                        uint32_t *result, size_t count);
void binary_and_vec_avx2(const uint32_t *a, const uint32_t *b,
                        uint32_t *result, size_t count);
void binary_or_vec_avx2(const uint32_t *a, const uint32_t *b,
                        uint32_t *result, size_t count);

// 8-bit operations (32 elements at a time)
void binary_add_vec_avx2_u8(const uint8_t *a, const uint8_t *b,
                           uint8_t *result, size_t count);
void binary_xor_vec_avx2_u8(const uint8_t *a, const uint8_t *b,
                           uint8_t *result, size_t count);

// 64-bit operations (4 elements at a time)
void binary_add_vec_avx2_u64(const uint64_t *a, const uint64_t *b,
                            uint64_t *result, size_t count);
void binary_xor_vec_avx2_u64(const uint64_t *a, const uint64_t *b,
                            uint64_t *result, size_t count);

// Popcount
uint32_t popcount_vec_avx2(const uint32_t *data, size_t count);
```

Scalar Fallbacks (always available)

```
void binary_add_vec_scalar(const uint32_t *a, const uint32_t *b,
                           uint32_t *result, size_t count);
void binary_xor_vec_scalar(const uint32_t *a, const uint32_t *b,
                           uint32_t *result, size_t count);
```

Example Usage

```
#include "bci/bci_simd.h"

const size_t SIZE = 1024;
uint32_t *a = malloc(SIZE * sizeof(uint32_t));
uint32_t *b = malloc(SIZE * sizeof(uint32_t));
uint32_t *result = malloc(SIZE * sizeof(uint32_t));

// Initialize arrays
for (size_t i = 0; i < SIZE; i++) {
    a[i] = i;
    b[i] = i * 2;
}

// Use AVX2 if available, otherwise scalar
if (bci_has_avx2_support()) {
    binary_add_vec_avx2(a, b, result, SIZE);
    binary_xor_vec_avx2(a, b, result, SIZE);
} else {
    binary_add_vec_scalar(a, b, result, SIZE);
    binary_xor_vec_scalar(a, b, result, SIZE);
}

free(a);
free(b);
free(result);
```

BTL (Boolean Translation Layer)

ISA Support (btl_isa.h)

Multi-architecture instruction set support with opcode decoding.

Enumerations

```
typedef enum {
    BTL_ARCH_X86_64,
    BTL_ARCH_ARM64,
    BTL_ARCH_RISCV,
    BTL_ARCH_UNKNOWN
} BTL_Architecture;

typedef enum {
    BTL_CAT_ARITHMETIC,
    BTL_CAT_LOGIC,
    BTL_CAT_SHIFT,
    BTL_CAT_MEMORY,
    BTL_CAT_CONTROL,
    BTL_CAT_SIMD,
    BTL_CAT_SYSTEM,
    BTL_CAT_UNKNOWN
} BTL_InstructionCategory;
```

Data Structures

```
typedef struct {
    const char *mnemonic;
    uint32_t opcode;
    BTL_InstructionCategory category;
    uint8_t operand_count;
    const char *description;
} BTL_InstructionDescriptor;
```

Functions

```
// Architecture detection
BTL_Architecture btl_detect_architecture(void);
const char* btl_architecture_name(BTL_Architecture arch);

// x86-64 ISA
const BTL_InstructionDescriptor* btl_x86_64_get_instruction(uint8_t opcode);
const char* btl_x86_64_get_mnemonic(uint8_t opcode);
BTL_InstructionCategory btl_x86_64_get_category(uint8_t opcode);

// ARM64 ISA (basic support)
const BTL_InstructionDescriptor* btl_arm64_get_instruction(uint32_t opcode);
const char* btl_arm64_get_mnemonic(uint32_t opcode);
BTL_InstructionCategory btl_arm64_get_category(uint32_t opcode);

// RISC-V ISA (basic support)
const BTL_InstructionDescriptor* btl_riscv_get_instruction(uint32_t opcode);
const char* btl_riscv_get_mnemonic(uint32_t opcode);
BTL_InstructionCategory btl_riscv_get_category(uint32_t opcode);
```

Example Usage

```
#include "btl/btl_isa.h"

// Detect current architecture
BTL_Architecture arch = btl_detect_architecture();
printf("Running on: %s\n", btl_architecture_name(arch));

// Decode x86-64 instruction
const BTL_InstructionDescriptor *inst = btl_x86_64_get_instruction(0x01);
printf("Opcode 0x01: %s (%s)\n", inst->mnemonic, inst->description);

// Get instruction category
BTL_InstructionCategory cat = btl_x86_64_get_category(0x89);
if (cat == BTL_CAT_MEMORY) {
    printf("Memory operation\n");
}
```

Register Allocator (btl_regalloc.h)

Linear scan register allocation with spilling support.

Data Structures

```
typedef enum {
    BTL_REG_GENERAL,
    BTL_REG_FLOAT,
    BTL_REG_VECTOR,
    BTL_REG_SPECIAL
} BTL_RegisterType;

typedef struct {
    uint32_t var_id;
    uint32_t start;
    uint32_t end;
    int assigned_register;
    bool spilled;
    uint32_t spill_location;
} BTL_LiveInterval;
```

Functions

```
// Allocator lifecycle
BTL_RegAllocator* btl_regalloc_create(size_t num_registers, BTL_RegisterType type);
void btl_regalloc_destroy(BTL_RegAllocator *allocator);

// Register operations
int btl_regalloc_acquire(BTL_RegAllocator *allocator, uint32_t var_id,
                        uint32_t start, uint32_t end);
void btl_regalloc_release(BTL_RegAllocator *allocator, int reg_id);
bool btl_regalloc_is_available(BTL_RegAllocator *allocator, int reg_id);

// Live interval management
void btl_regalloc_add_interval(BTL_RegAllocator *allocator, uint32_t var_id,
                              uint32_t start, uint32_t end);
const BTL_LiveInterval* btl_regalloc_get_interval(BTL_RegAllocator *allocator,
                                                  uint32_t var_id);

// Linear scan allocation
bool btl_regalloc_linear_scan(BTL_RegAllocator *allocator);

// Spilling
bool btl_regalloc_needs_spill(BTL_RegAllocator *allocator);
uint32_t btl_regalloc_get_spill_count(BTL_RegAllocator *allocator);

// Statistics
size_t btl_regalloc_get_allocated_count(BTL_RegAllocator *allocator);
size_t btl_regalloc_get_free_count(BTL_RegAllocator *allocator);

// Register naming
const char* btl_regalloc_get_register_name(int reg_id, BTL_RegisterType type);
```

Example Usage

```
#include "btl/btl_regalloc.h"

// Create allocator with 16 general-purpose registers
BTL_RegAllocator *alloc = btl_regalloc_create(16, BTL_REG_GENERAL);

// Add live intervals for variables
btl_regalloc_add_interval(alloc, 1, 0, 10); // var1: [0, 10]
btl_regalloc_add_interval(alloc, 2, 5, 15); // var2: [5, 15]
btl_regalloc_add_interval(alloc, 3, 12, 20); // var3: [12, 20]

// Perform linear scan allocation
if (btl_regalloc_linear_scan(alloc)) {
    printf("Allocation successful\n");

    if (btl_regalloc_needs_spill(alloc)) {
        printf("Spills required: %u\n", btl_regalloc_get_spill_count(alloc));
    }
}

btl_regalloc_destroy(alloc);
```

Instruction Scheduler (btl_scheduler.h)

Dependency-aware instruction scheduling using list scheduling algorithm.

Functions

```
// Scheduler lifecycle
BTL_Scheduler* btl_scheduler_create(void);
void btl_scheduler_destroy(BTL_Scheduler *scheduler);

// Add instructions
uint32_t btl_scheduler_add_instruction(BTL_Scheduler *scheduler,
                                       uint32_t opcode, uint32_t latency);

// Add dependencies
void btl_scheduler_add_dependency(BTL_Scheduler *scheduler,
                                  uint32_t from_id, uint32_t to_id);

// Build and schedule
bool btl_scheduler_build_graph(BTL_Scheduler *scheduler);
bool btl_scheduler_schedule(BTL_Scheduler *scheduler);

// Get results
const uint32_t* btl_scheduler_get_schedule(BTL_Scheduler *scheduler,
                                             size_t *out_count);
uint32_t btl_scheduler_get_critical_path(BTL_Scheduler *scheduler);

// Statistics
size_t btl_scheduler_get_instruction_count(BTL_Scheduler *scheduler);
uint32_t btl_scheduler_get_total_latency(BTL_Scheduler *scheduler);
```

Example Usage

```
#include "btl/btl_scheduler.h"

BTL_Scheduler *sched = btl_scheduler_create();

// Add instructions with latencies
uint32_t id1 = btl_scheduler_add_instruction(sched, 0x01, 2); // ADD, 2 cycles
uint32_t id2 = btl_scheduler_add_instruction(sched, 0x89, 1); // MOV, 1 cycle
uint32_t id3 = btl_scheduler_add_instruction(sched, 0x31, 1); // XOR, 1 cycle

// Add dependencies (id1 must complete before id2)
btl_scheduler_add_dependency(sched, id1, id2);
btl_scheduler_add_dependency(sched, id2, id3);

// Schedule instructions
if (btl_scheduler_schedule(sched)) {
    size_t count;
    const uint32_t *schedule = btl_scheduler_get_schedule(sched, &count);

    printf("Scheduled %zu instructions\n", count);
    printf("Critical path: %u cycles\n",
           btl_scheduler_get_critical_path(sched));
}

btl_scheduler_destroy(sched);
```

Peephole Optimizer (btl_peekhole.h)

Pattern-based peephole optimization with predefined rules.

Data Structures

```
typedef struct {
    uint32_t opcode;
    uint32_t operand1;
    uint32_t operand2;
    bool wildcard_op1;
    bool wildcard_op2;
} BTL_InstructionPattern;

typedef struct {
    const char *name;
    BTL_InstructionPattern *pattern;
    size_t pattern_length;
    BTL_InstructionPattern *replacement;
    size_t replacement_length;
    uint32_t savings;
} BTL_OptimizationRule;
```

Functions

```
// Optimizer lifecycle
BTL_PeepholeOptimizer* btl_peekhole_create(void);
void btl_peekhole_destroy(BTL_PeepholeOptimizer *optimizer);

// Add rules
bool btl_peekhole_add_rule(BTL_PeepholeOptimizer *optimizer,
                           const BTL_OptimizationRule *rule);

// Apply optimizations
size_t btl_peekhole_optimize(BTL_PeepholeOptimizer *optimizer,
                              BTL_InstructionPattern *instructions,
                              size_t count,
                              BTL_InstructionPattern *output,
                              size_t output_capacity);

// Statistics
uint32_t btl_peekhole_get_total_savings(BTL_PeepholeOptimizer *optimizer);
size_t btl_peekhole_get_rules_applied(BTL_PeepholeOptimizer *optimizer);
```

Predefined Rules

```
extern const BTL_OptimizationRule BTL_RULE_REDUNDANT_MOVE;
extern const BTL_OptimizationRule BTL_RULE_STRENGTH_REDUCTION;
extern const BTL_OptimizationRule BTL_RULE_CONSTANT_FOLDING;
extern const BTL_OptimizationRule BTL_RULE_DEAD_CODE;
```

Example Usage

```
#include "btl/btl_peekhole.h"

BTL_PeekholeOptimizer *opt = btl_peekhole_create();

// Add optimization rules
btl_peekhole_add_rule(opt, &BTL_RULE_REDUNDANT_MOVE);
btl_peekhole_add_rule(opt, &BTL_RULE_STRENGTH_REDUCTION);
btl_peekhole_add_rule(opt, &BTL_RULE_CONSTANT_FOLDING);

// Prepare instruction sequence
BTL_InstructionPattern input[100];
BTL_InstructionPattern output[100];

// ... populate input ...

// Apply optimizations
size_t count = btl_peekhole_optimize(opt, input, 100, output, 100);

printf("Optimized to %zu instructions\n", count);
printf("Rules applied: %zu\n", btl_peekhole_get_rules_applied(opt));
printf("Cycle savings: %u\n", btl_peekhole_get_total_savings(opt));

btl_peekhole_destroy(opt);
```

Build System

CMake Configuration

```
mkdir build && cd build
cmake ..
make
make test
```

Build Options

```
# Enable sanitizers
cmake -DENABLE_ASAN=ON ..    # AddressSanitizer
cmake -DENABLE_UBSAN=ON ..   # UndefinedBehaviorSanitizer
cmake -DENABLE_MSAN=ON ..    # MemorySanitizer

# Release build with optimizations
cmake -DCMAKE_BUILD_TYPE=Release ..
```

Feature Detection

The build system automatically detects:

- AVX2 support (HAVE_AVX2)
- 128-bit integer support (HAVE_INT128)
- C23 features

Testing

Running Tests

```
# Run all tests
make test

# Run specific test
./test_bci_arithmetic
./test_btl_regalloc
```

Test Coverage

- **BCI Tests:** 200+ unit tests
 - test_bci_arithmetic : 50+ tests
 - test_bci_bitops : 60+ tests
 - test_bci_conversion : 50+ tests
 - test_bci_simd : 40+ tests
 - **BTL Tests:** 300+ unit tests
 - test_btl_isa : 80+ tests
 - test_btl_regalloc : 70+ tests
 - test_btl_scheduler : 80+ tests
 - test_btl_peephole : 70+ tests
-

Performance

Running Benchmarks

```
./benchmark_bci
./benchmark_btl
```

Expected Performance

BCI Benchmarks:

- binary_add_u64 : ~500 Mops/s
- popcount_u64 : ~800 Mops/s
- clz_u64 : ~1000 Mops/s
- AVX2 vectorized operations: 4-8x speedup over scalar

BTL Benchmarks:

- Linear scan allocation (100 variables): <1ms
 - List scheduling (100 instructions): <2ms
 - Peephole optimization (1000 instructions): <5ms
-

Risk Mitigation

Feature Detection Macros

```
#if defined(__STDC_VERSION__) && __STDC_VERSION__ >= 202311L
    // C23 features available
#endif

#ifdef __SIZEOF_INT128__
    // 128-bit integers available
#endif

#if defined(__AVX2__)
    // AVX2 intrinsics available
#endif
```

Sanitizer Support

All code is tested with:

- AddressSanitizer (memory errors)
- UndefinedBehaviorSanitizer (undefined behavior)
- MemorySanitizer (uninitialized memory)

API Contracts

- All pointer parameters are checked for NULL
- Buffer sizes are validated
- Return values indicate success/failure
- Overflow/underflow is detected and reported

License

Part of the Binary Decomposition Interface (BDI) project.